

# SENG 300 Server and Network

The goal of this document is to provide all the documentation potentially needed to use and work with a database server. This document explains the general workings of the server, network, and database systems. A basic understanding of computer networking principles and cryptology is assumed, but I have tried to provide a simplified explanation of some sections for those without. Generally speaking, understanding the TLDR sections is likely enough for most contributors; additional explanations are only provided for curiosity and possible edge cases, so feel free to skip directly to the TLDR sections. Throughout this doc, the users' computers may be referred to as "clients", while the actual database is referred to as the "server".

## Dependencies:

The server computer must have SQLite and a JDBC driver installed. Minor changes to default Windows settings are also required. The client computers require nothing extra.

## Overview:

On the server, a single port is opened, and the computer constantly listens for TCP/IP connections from clients. Currently on the test server, port 14001 is being used. This can be changed if necessary.

For clients, the server computer is located and connected to using the server's internal IP address and the port number (in this case, port 14001). Connections are always initiated by clients.

Once a connection has been made, data is transferred between the two in the form of a stream of bits, which are interpreted by the computers as a byte array. These transmissions are unencrypted and visible to other devices on the network, so for security, a basic encryption system has been implemented.

Connections are held indefinitely. Once a connection has been made, it is held until the client closes their program. Each client connection gets one Session thread, and the server can handle multiple sessions at a time. All Sessions can request data at the same time, but only a single Session is allowed to edit the data at a time.

## TLDR:

Currently, there are two versions of the program: One for the clients (i.e. users) and one for a dedicated server. All of the information for the platform (usernames, passwords, game info, etc...) are held on the server. When the client needs to request/update information, a request is made to the server over the network connection, and the necessary data is transferred.

# Encryption and Message Types:

The first time a client computer connects to the server, a new client ID is assigned to the client. Client ID's are tied to the client computers, not the account(s) using that computer. Client ID's are not stored in persistent memory and must be regenerated with every connection.

After assigning a new client ID, an encryption key is generated for use between the server and the specific client. First, a shared secret known only to the client and server is created using the Diffie-Hellman Protocol. This shared secret is then used to generate an encryption key for the AES algorithm on GCM mode. On the server side, these keys are stored alongside the associated client IDs in a HashTable for later retrieval and saved in the Session threads. Client IDs are only used for establishing encryption keys.

The most notable feature of AES-GCM encryption is the inclusion of nonces. Normally, when two of the plaintexts are encrypted, the same ciphertext is produced. This is generally considered undesirable, as it allows attackers to pattern match and recognise when the same bits/transmissions are being sent. Nonces (sometimes also referred to as IV, which used to perform similar roles in now outdated AES modes) are randomly generated bits added to every message before AES encryption is performed, so as to ensure that two identical plaintexts will not produce the same ciphertexts. For decryption purposes, the nonce has to be passed along and declared to the other party, so the first 12 bytes of every encrypted transmission includes a copy of the nonce.

Beyond that, when transmitting raw bytes over the network, the other party has to know where each message starts and ends, so they know when to stop/start listening. In the current setup, each message is preceded by 4 bytes, which indicate the length of the actual message.

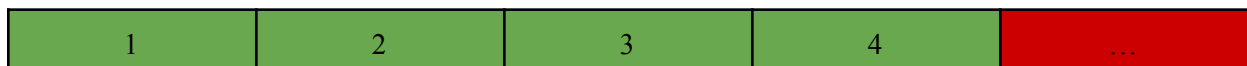
Here are the different message types, what they would look like, and a brief description of when they would be used:

*Description:*



A singular unencrypted byte sent before every transmission that tells the other party how to interpret the next transmission. There are 255 possible values for this, with each one representing a different request (not all 255 values are used). For example, a description value of “1” means the next message will be a login request. No response from the other party is required for this type of message.

*ClientID:*



At the start of a connection, the client sends their client ID (or simply “new” if they don’t have one) before an encryption key is agreed upon. Because no key has been decided on, the message is unencrypted. The green bytes contain the number of red bytes, and red bytes represent the unencrypted clientID. Once the server receives this transmission, it will create a new key if necessary or otherwise use

a premade key. If the server uses a premade key, a single byte is sent with the value “1”. If the server has no premade key, a single byte is sent with the value “0”, and additional transmissions are made to create a new key.

*Request:*

|   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |     |
|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|-----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | ... |
|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|-----|

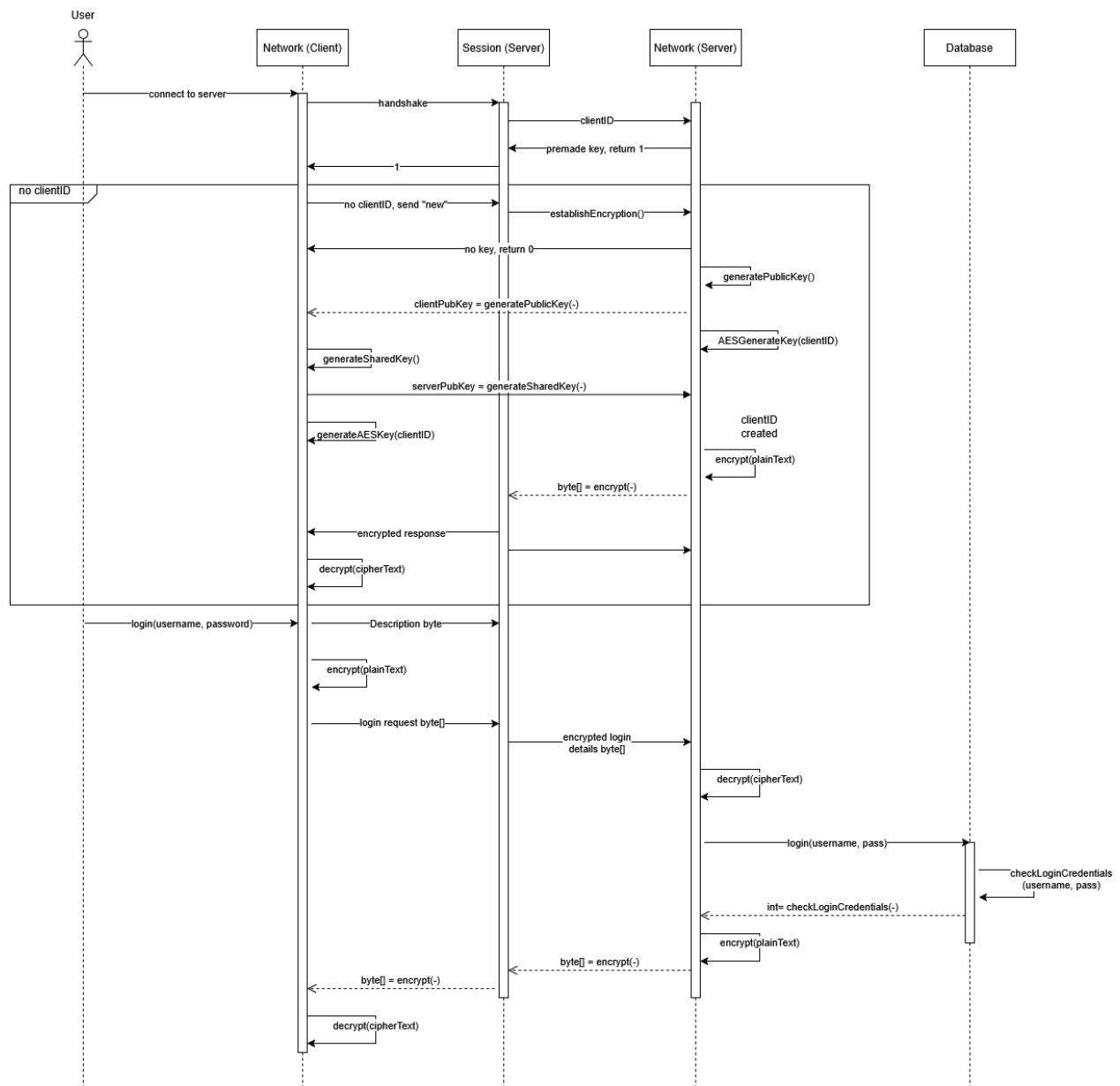
After an encryption key has been decided, requests can now be sent over the connection. These requests take the following form: The green bytes contain the number of blue and yellow bytes; The blue bytes contain the nonce; And the yellow bytes contain the encrypted contents of the request. The yellow bytes only contain the parameters of the request; The request itself is sent using the *Description* message sent immediately prior (see above). Each parameter is sent separately, and depending on the nature of the request, the other party may or may not continue the conversation with their own messages.

*Pings (UNUSED):*

|   |
|---|
| 1 |
|---|

Pings are a special subtype of *Descriptions* that are used to check status changes. Pings are single unencrypted bytes that contain values of 0. These bytes are repeatedly sent to the other party to check for status changes, such as a new chat message or an incoming match challenge. If no new status changes exist, the other party returns the ping with another ping. If a status change does occur, a normal *Description* message is sent along with the following *general* message. When pinging, pings are repeatedly sent at semi-regular intervals, with a minimum delay of 5 seconds between each ping.

Here is a sequence diagram illustrating the login process when a new client attempts to connect (credit to Sanmeet for providing the diagram):



Note: The initial clientID sent by the client is empty, and so the server creates and assigns the client a new one

## TLDR:

Honestly, there's no need to understand how encryption works. All you need to know from this -if anything all- is that messages are encrypted, and client IDs are tied to the computers, NOT user accounts.

## (Non)issues:

There are 2 main complications with this setup:

### 1) Before connecting to the server, clients must manually adjust their code to match the server's IP address

IP addresses are normally assigned by the router; Without knowing what IP is going to be assigned to your server, the clients cannot be configured to connect automatically. One partial workaround for this is using static IP address. IP addresses are normally randomly assigned by the router using the DHCP protocol. If you use a static IP for your server instead (either by changing your server's network settings or adjusting your router settings), you can give your server a fixed IP address that doesn't change. Now you will only have to adjust the client's code the very first time the server goes online, and never again (so long as you stay on the same network).

### 2) Both the server and client(s) must be on the same network

When a device sends out a packet into the internet, the router will replace the device's IP with a public IP (oftentimes the router's IP) using NAT. To everyone else on a different network, all they can see is the router and not any of the devices actually on it. When you try to directly connect to the server from a different network, the connection will fail because the actual server is "invisible". The IP address assigned to the server is an internal IP address, and only exists within that subnet. Outside of it, the IP is considered unroutable, and all routers will automatically drop packets meant for private IP addresses. Even if it were somehow possible for the packets to reach the server's router without getting dropped, routers still automatically refuse connections initiated from outside because of a lack of NAT mapping.

Both of these problems are only problems for the current school setup, and wouldn't actually be an issue for our fictional scenario. As a real company, with control over our company's internet subscriptions and router, both these problems could easily be solved. With a business-class internet subscription, it would be possible for our router to get a static IP, which would never change (normally, for a non-commercial internet subscription, routers are given random IP addresses by the ISP). With that, client programs could be hard-coded to connect to our router's specific IP address and would never have to manually edit the client-side code. At the same time, with access to our router, we could set up port forwarding to allow connections from other networks. As mentioned earlier, direct connections to devices from different networks are not possible. But it is possible to directly connect to the server's router, which *does* have a public IP address. Then, with port forwarding enabled on the router, traffic to the router can be redirected to the server, allowing connections from other networks.